

A tool for Formal Verification of Spiking Neural Networks via Weight-Discretized Quotient Abstractions

Anonymous submission¹

Anonymous Institution

Abstract. Spiking Neural Networks (SNNs) more closely model biological neural dynamics than classical artificial networks, offering energy-efficient computation but pose severe challenges for formal verification due to state space explosion. Existing quotient model abstractions reduce the state space by partitioning membrane potentials into equivalence classes, but discard synaptic weight information, limiting the properties that can be verified. This paper introduces a *weight-discretized quotient model abstraction* that maps continuous synaptic weights to a compact integer range while preserving the relative contribution of each synapse. The discretization is accompanied by formal correctness guarantees: a Threshold Preservation theorem (completeness) ensures that no fireable configurations are lost, and an Asymptotic Silence theorem (soundness) guarantees that no spurious spikes are introduced. The core biological properties of Leaky Integrate-and-Fire neurons—tonic spiking, integrator behaviour, and excitability—are provably maintained. A topology-dependent scaling analysis derives closed-form state space formulas and shows that the reduction compounds exponentially across neurons: approximately $17\times$ per neuron for a discretization parameter $W = 3$. Empirical validation across seven canonical topologies in the PRISM model checker confirms the theoretical predictions, enabling verification of networks that are otherwise intractable.

Keywords: Spiking Neural Networks · Formal Verification · Model Checking · PRISM · Quotient Abstraction · Weight Discretization

1 Introduction

The evolution of artificial neural networks is conventionally classified into three generations [1]. The first generation, epitomized by McCulloch–Pitts threshold gates, employed binary inputs and outputs and was limited to linearly separable problems. The second generation introduced continuous activation functions

and gradient-based optimization via backpropagation, enabling deep learning architectures that have achieved remarkable success in pattern recognition and sequence modelling. Yet these networks lack the temporal dynamics and energy efficiency of biological neural circuits.

The third generation—Spiking Neural Networks (SNNs)—bridges the gap between machine learning and computational neuroscience. SNNs employ biologically plausible neurons communicating via discrete, asynchronous spikes, where the precise timing of each spike conveys information alongside the aggregate firing rate [1]. This event-driven computation offers substantial power advantages, making SNNs suitable for neuromorphic hardware [2]. However, the non-differentiable Heaviside activation function precludes standard gradient-based optimization, and the stochastic, temporal nature of spike-based computation makes formal verification of correctness significantly harder than for traditional networks. As SNNs are increasingly targeted for safety-critical applications, formal methods providing mathematical guarantees of functional correctness become essential.

This paper presents a novel probabilistic verification framework for spiking neural networks: a *weight-discretized quotient model abstraction* for the formal verification of SNNs via probabilistic model checking. The approach extends the filtration-based quotient model of Baier and Katoen [3] to preserve synaptic weight information during abstraction, enabling verification in the PRISM model checker [4] with drastically reduced state spaces. The contributions are threefold:

1. A **weight discretization scheme** that maps continuous synaptic weights to a finite discrete range while preserving threshold feasibility and relative synaptic contributions (Section 4.1).
2. **Formal correctness proofs:** a *Threshold Preservation* theorem (completeness) and an *Asymptotic Silence* theorem (soundness) guaranteeing that the discretization neither loses fireable configurations nor introduces spurious spikes (Section 4.4).
3. A **topology-dependent state space analysis** with closed-form formulas for DTMC size as a function of network structure, validated empirically across seven canonical topologies (Section 6).

2 Related Work

2.1 Formal Verification of SNNs

De Maria et al. [5] pioneered the modelling of SNNs as timed automata, formalizing Leaky Integrate-and-Fire (LIF) neurons with parameter learning. Their work established key biological properties—tonic spiking, integrator behaviour, excitability—as formal verification targets, and introduced the Advice Back-Propagation (ABP) algorithm for supervised parameter inference driven by model-checking counter-examples rather than continuous gradients. This approach was extended to *neuronal archetypes*—primitive micro-circuits such as contralateral inhibition and convergent excitation—allowing macroscopic network properties to be composed from formally verified building blocks [6].

Yao et al. [7] introduced the Refractory-evolve Probabilistic LI&F (RP-LI&F) neuron model, unifying discrete-time refractory dynamics with probabilistic spike generation. Their contract-based verification approach translates SNN topologies into Discrete-Time Markov Chains (DTMCs) and specifies behavioural properties using Probabilistic Computation Tree Logic (PCTL), enabling rigorous assume/guarantee contracts.

These approaches establish the feasibility of formal SNN verification but share a common limitation: the *state space explosion problem*. As network size grows, the DTMC state space grows exponentially in the number of neurons, and existing quotient model abstractions [3] lose synaptic weight information during reduction. The present work addresses both limitations.

The present work addresses these limitations by introducing a weight-discretized quotient model abstraction that preserves synaptic weight information while drastically reducing the state space, and by providing a unified tool—CogSpike—that integrates both simulation and formal verification within a single workbench.

3 Preliminaries

3.1 Spiking Neural Network Model

An SNN is modelled as a directed graph $G = (V, E)$, where directed edges represent unidirectional synaptic connections and $V = V_{\text{in}} \cup V_{\text{proc}}$ partitions into

input neurons and processing neurons, and $E \subseteq V \times V$ represents directed synaptic connections with integer weights $w_e \in [-100, 100]$.

Each processing neuron $n \in V_{\text{proc}}$ follows Leaky Integrate-and-Fire (LIF) dynamics [5, 8]. Let $\ell \in [0, 1]$ be the leak factor and P_{rth} be the firing threshold. At each discrete time step t , the membrane potential integrates incoming weighted spikes and decays toward rest:

$$p_n(t+1) = \max \left(0, (1-\ell) \cdot p_n(t) + \sum_{i \in \text{In}(n)} w_{i,n} \cdot y_i(t) \right) \quad (1)$$

where $y_i(t)$ is the spike event of presynaptic neuron i at time t . When $p_n(t+1) \geq P_{\text{rth}}$, neuron n emits a spike ($y_n(t+1) = 1$) and resets to zero.

Firing is *probabilistic*: the potential maps to a discrete integer threshold level $L \in \{0, \dots, N-1\}$ where N is configurable (1–10), and each level yields a firing probability. Optionally, neurons implement a three-state refractory machine: Normal ($s = 0$), Absolute Refractory Period (ARP, $s = 1$), and Relative Refractory Period (RRP, $s = 2$), with reduced firing probability during RRP scaled by factor α .

3.2 Probabilistic Model Checking and Temporal Logics

Definition 1. A Discrete-Time Markov Chain (DTMC) is a tuple $\mathcal{D} = (S, s_0, \mathbf{P})$ where S is a finite set of states, $s_0 \in S$ is the initial state, and $\mathbf{P} : S \times S \rightarrow [0, 1]$ is the transition probability matrix satisfying $\sum_{s' \in S} \mathbf{P}(s, s') = 1$ for all $s \in S$ [3].

Behavioural properties to be verified are specified in Probabilistic Computation Tree Logic (PCTL), which extends branching-time temporal logic with probabilistic path quantifiers $P_{\bowtie p}[\varphi]$. For instance, $P_{\geq 1}[\mathbf{F}(y_n = 1)]$ asserts that neuron n fires with probability one, while $P_{\geq 1}[\mathbf{G}(y_n = 0)]$ asserts permanent silence.

PRISM [4] is a probabilistic model checker supporting DTMCs. Models are specified as parallel compositions of *modules* with local integer variables and guarded probabilistic transitions, synchronizing via shared labels. The global state space is the Cartesian product of all module state spaces. PRISM offers an *explicit* engine (enumerates reachable states) and a *symbolic BDD* engine (uses Binary Decision Diagrams via the CUDD library), whose efficiency depends on variable ordering.

3.3 Quotient Model Abstraction

Quotient model abstraction [3, 9] reduces the DTMC state space by partitioning states into equivalence classes. The filtration-based approach maps continuous membrane potentials to a finite class set, preserving probabilistic branching structure. However, the standard filtration treats all synapses uniformly, collapsing weight information: it cannot distinguish strong excitatory from weak excitatory or inhibitory contributions. The weight discretization scheme presented in Section 4 addresses this limitation.

4 Weight-Discretized Quotient Abstraction

The standard filtration-based quotient model abstracts membrane potentials into equivalence classes but treats all synapses uniformly, discarding weight information. This section introduces a *weight discretization scheme* that preserves the relative contribution of each synapse.

4.1 Weight Discretization Function

Definition 2. *Given a weight range $[-w_{\max}, w_{\max}]$ (typically $w_{\max} = 100$) and a discretization parameter $W \in \mathbb{N}^+$, the weight discretization function $\delta_W : \mathbb{R} \rightarrow \mathbb{Z}$ is:*

$$\delta_W(w) = \text{round}\left(w \cdot \frac{W}{w_{\max}}\right) = \left\lfloor w \cdot \frac{W}{w_{\max}} \right\rfloor \quad (2)$$

mapping original weights to the discrete range $[-W, W] \subset \mathbb{Z}$.

The function preserves relative weight magnitudes: for $W = 3$, strong excitatory ($w = 100$) maps to $\delta_3(100) = 3$, medium ($w = 67$) to $\delta_3(67) = 2$, weak ($w = 33$) to $\delta_3(33) = 1$, and inhibitory ($w = -50$) to $\delta_3(-50) = -2$. Weights are stored as static integer constants in the generated PRISM model, contributing no additional state variables.

The *weighted contribution* for neuron n with discretized incoming weights $\{w_1^d, \dots, w_m^d\}$ replaces the binary class evolution of the original quotient model:

$$C_n = \sum_{i=1}^m w_i^d \cdot y_i \quad (3)$$

where $y_i \in \{0, 1\}$ is the spike output of presynaptic neuron i and m is the fan-in.

4.2 Threshold Calibration

Definition 3. *The discretized threshold for a neuron with original threshold T is:*

$$T_d = \left\lceil T \cdot \frac{W}{w_{\max}} \right\rceil \quad (4)$$

The use of the ceiling function $\lceil \cdot \rceil$ (rather than rounding) ensures $T_d \geq T \cdot W/w_{\max}$, so the discretized neuron is *at least as hard* to fire as the original. This conservative calibration prevents false-positive firings and is essential for the soundness guarantee established in Section .

4.3 Threshold-Dependent Leak Factor

Definition 4. *The discretized leak factor is:*

$$\lambda_d = -\max(1, \lfloor \ell \cdot T_d \rfloor) \quad (5)$$

where $\ell \in [0, 1]$ is the leak rate ($\ell = 1 - r$ with retention rate r).

This formulation links the leak to T_d rather than to the number of equivalence classes k . Since T_d scales with the actual potential range, the decay remains proportional to realistic membrane dynamics. The $\max(1, \dots)$ floor ensures a minimum decay of one unit per step, which is required by the Soundness theorem (Section). The class evolution becomes:

$$c'_n = \text{clamp}(c_n + \Delta(C_n) + \lambda_d, 0, k) \quad (6)$$

where $\Delta(C_n) = \text{clamp}(\lfloor C_n/\gamma \rfloor, -k, k)$ is the class delta function with class width $\gamma = T_d/k$.

4.4 Formal Proofs

This subsection presents the formal proofs of the key correctness properties. For complete derivations, the reader is referred to the extended appendix.

Threshold Preservation (Completeness).

Theorem 5. *Let $\mathcal{N}$ be a neuron with weights $\{w_1, \dots, w_m\}$ and threshold T . If $\mathcal{N}$ can fire in a single step (i.e., $\exists \mathbf{y} \in \{0, 1\}^m$ such that $\sum_{i=1}^m w_i \cdot y_i \geq T$), then*

the discretized neuron $\mathcal{N}'$ with weights $\{\delta_W(w_1), \dots, \delta_W(w_m)\}$ and threshold T_d can also fire.

Proof. Let $\mathbf{y}^*$ be a firing input pattern with weighted sum $S = \sum w_i y_i^* \geq T$. By the rounding bound, $\delta_{W(w_i)} \geq w_i \cdot W / w_{\max} - 1/2$, so the discretized sum satisfies:

$$S_d = \sum \delta_{W(w_i)} \cdot y_i^* \geq S \cdot W / w_{\max} - m^*/2 \quad (7)$$

where $m^* = \sum y_i^* \leq m$ is the number of active inputs. Since $T_d = \lceil T \cdot W / w_{\max} \rceil \leq T \cdot W / w_{\max} + 1$, the sufficient condition for firing is $(S - T) \cdot W / w_{\max} \geq m/2 + 1$. For $W \geq w_{\max} \cdot (m/2 + 1)/T$, the condition is satisfied whenever $S \geq T$. With typical parameters ($W = 3$, $w_{\max} = 100$, $m \leq 10$, $T = 100$), the bound requires $W \geq 6$. $\square$

Remark 6. The cumulative rounding error of $-m/2$ implies that high fan-in neurons require finer discretization. For neurons with $m > 2W$, the rounding noise may exceed the smallest synaptic weight, and a threshold correction factor $T'_d = T_d - \lfloor m/(2W) \rfloor$ can optionally be applied.

Asymptotic Silence (Soundness).

Theorem 7. Let $\mathcal{N}'$ be a discretized neuron with potential $P_t < T_d$ and leak factor $\lambda_d \leq -1$. If the input is zero for all $t' \geq t$ (i.e., $S_d = 0$ henceforth), then $\mathcal{N}'$ will never fire.

Proof. Without input, $P_{t+1} = \max(0, P_t + \lambda_d)$. Since $\lambda_d \leq -1$ and $P_t > 0$, the potential strictly decreases: $P_{t+1} \leq P_t - 1$. The sequence converges to the absorbing state $P = 0$. Since the trajectory is non-increasing and starts below T_d , the firing condition $P \geq T_d$ is never met. At $P = 0$, the threshold level is $L = 0$, which maps to firing probability zero, ensuring permanent silence. $\square$

Biological Property Preservation.

The discretized model preserves the core LIF properties formalized by De Maria et al. [5]:

- **Tonic spiking.** Under constant input C_{in} , the neuron has non-zero firing probability iff $C_{\text{in}} > |\lambda_d|$. When satisfied, the potential accumulates to a level $L > 0$, yielding periodic probabilistic spiking with expected inter-spike interval $\text{ISI} = \lceil T_d / (C_{\text{in}} - |\lambda_d|) \rceil$.

- **Integrator.** The probability of immediate firing on simultaneous inputs reaches 1.0 iff $\sum \delta_{W(w_i)} \geq T_d$. Below threshold, the response is graded via the threshold level mapping.
- **Excitability.** The expected inter-spike interval decreases monotonically as input strength increases, since $\text{ISI} = \lceil T_d / (C_{\text{in}} - |\lambda_d|) \rceil$ is a decreasing function of C_{in} .

5 The CogSpike Workbench

Existing SNN simulation platforms—Nengo [10], Brian 2 [11], NEST [12] — provide rich environments for designing and simulating spiking neural networks but lack integrated support for *formal verification*. Conversely, probabilistic model checkers such as PRISM [4] operate on abstract models that must be constructed and maintained separately from the simulation code, creating an isomorphism gap between the system under study and its formal representation.

CogSpike bridges this gap by unifying SNN *design*, *simulation*, and *verification* in a single desktop workbench. The tool is implemented in Rust with an immediate-mode GUI (egui), and its core design principle is **strict isomorphism**: the simulation engine and the PRISM model generator share the same mathematical model—the LIF dynamics of Equation 1, the three-state refractory machine, and the probabilistic firing logic—so that verification results faithfully predict simulation behaviour.

The workbench provides:

1. A **visual graph editor** for constructing SNN topologies as directed graphs with configurable synaptic weights, input spike generators (periodic, Poisson, burst, custom), and multi-generator combination modes (OR, AND, XOR).
2. An **isomorphic simulation engine** that executes the LIF dynamics with configurable model complexity: three presets—Deterministic (1 threshold level, no refractory), Fast (4 levels, no refractory), and Full (10 levels, ARP/RRP enabled)—allow trading biological fidelity for computational tractability. Results are visualized via raster plots, membrane potential traces, and aggregate firing statistics.
3. **Automated PRISM code generation** that translates the SNN graph into a DTMC model. The generator supports both *precise* and *weight-discretized quotient* abstraction modes (Section 4), and synthesizes PCTL properties for reachability, safety, and liveness verification. Per-neuron potential bounds are computed from fan-in analysis to minimize the state space.
4. A **verification bridge** that invokes the PRISM model checker as a background process with configurable engine selection (explicit, sparse, MTBDD),

JVM memory limits, and solver options (Gauss–Seidel, topological value iteration). Results are parsed and displayed inline alongside the simulation output.

Since model complexity presets (Deterministic, Fast, Full) are shared between simulation and verification, any configuration tested in simulation can be directly verified, enabling a rapid *design–simulate–verify* workflow without manually constructing PRISM models.

6 Topology-Dependent Scaling Limits

To characterize the practical limits of SNN verification, this section derives closed-form formulas for the DTMC state space as a function of network topology and model configuration.

6.1 PRISM Module Decomposition

The PRISM model for an SNN $G = (V, E)$ consists of four module types:

- **GlobalClock**: a step counter with $T_{\max} + 1$ states.
- **Inputs**: $2^{|V_{\text{in}}|}$ states (one binary variable per input neuron).
- **Neuron** M_n : per-neuron state depends on configuration:
 - *Fast* ($k = 10$ threshold levels, no refractory): $|S_n^{\text{fast}}| = 2 \cdot R_n$ where $R_n = P_{\max, n} - P_{\min, n} + 1$.
 - *Full* ($k = 10$, ARP=2, RRP=4): $|S_n^{\text{full}}| = 3 \cdot 3 \cdot 5 \cdot 2 \cdot R_n = 90 \cdot R_n$.
 - *Discretized* ($W = 3$): $|S_n^{\text{disc}}| = 2 \cdot (P_{\max, n}^d + 1)$ where $P_{\max, n}^d = T_d + E_n^d$.
- **Transfer** $T_{i,j}$: 2 states per internal edge (representing the intermediate state of a spike traveling along a synapse).

Theorem 8. (State Space Product). *The theoretical state space is the Cartesian product of all module state spaces:*

$$|S_{\text{theory}}| = (T_{\max} + 1) \cdot 2^{|V_{\text{in}}|} \cdot \prod_{n \in V_{\text{proc}}} f_n(C) \cdot 2^{|E_{\text{int}}|} \quad (8)$$

where $f_n(C) = |S_n|$ depends on the model configuration C . The $2^{|E_{\text{int}}|}$ factor accounts for the binary states of all internal transfer edges.

For a chain of N neurons with $P_{\text{rth}} = 100$ and weight $w = 80$: the per-neuron factor is $f_n = 2 \cdot 121 = 242$ (fast precise), dropping to $f_n = 2 \cdot 7 = 14$ (discretized $W = 3$). The per-neuron reduction factor is $242/14 \approx 17.3 \times$.

Theorem 9. (*Exponential State Space Reduction*). *For a chain of N neurons, the state space ratio between precise and discretized models compounds exponentially:*

$$\frac{|S^{\text{precise}}|}{|S^{\text{disc}}|} = \prod_{n=1}^N \frac{R_n}{P_{\max,n}^d + 1} \approx 17.3^N \quad (9)$$

The factor 17.3 is the per-neuron reduction ratio $242/14$ from the chain example above. For $N = 4$: $17.3^4 \approx 89\{, \}500 \times$.

6.2 Empirical Validation

63 PRISM models were generated across 7 canonical topologies (single, chain-2 through chain-4, fork, diamond, convergent), 3 configurations (Det: deterministic, Fast: no refractory period, Full: complete refractory dynamics), and 3 model types (precise, disc. $W = 2$, disc. $W = 3$). Fig. 1 reports the reachable state counts from DTMC exports.

Fig. 1. Reachable DTMC states $|S_{\text{reachable}}|$ by topology and configuration. All precise models use $k = 10$ threshold levels. “OOM” denotes CUDD BDD out-of-memory (2 GB limit).

Topology	Det.	Fast	Full	Disc. $W=3$
Single (1N)	5	6	18	6
Chain-2 (2N)	12	23	246	11
Chain-3 (3N)	23	275	13,180	28
Chain-4 (4N)	42	6,917	362,289	88
Fork (3N)	12	83	3,388	15
Diamond (5N)	15	1,511	OOM	98
Convergent (1N)	17	20	65	16

The most striking pattern is the exponential growth across chain lengths in the precise configurations: from 18 states (single, full) to 362,289 (chain-4, full)—a $20\{, \}000 \times$ increase for just three additional neurons. The discretized column remains remarkably stable, growing from 6 to 88 across the same range.

The per-neuron multiplicative factor for the fast precise model is:

$$r_{\text{fast}} \approx 23/6 \approx 3.8, \quad 275/23 \approx 12.0, \quad 6917/275 \approx 25.2 \quad (10)$$

This acceleration occurs because downstream neurons receive progressively richer input distributions, approaching the theoretical maximum of 484 per neuron as the reachability density ρ increases.

Three further observations emerge: (i) *fan-out is more expensive than fan-in*—the fork topology (3 neurons, fan-out 2) produces 275 fast states versus 20 for the convergent topology (fan-in 2), because parallel branches multiply module counts; (ii) *BDD memory is the practical bottleneck*—the diamond topology (5 neurons) causes CUDD OOM in the full precise configuration, while the explicit engine handles chain-4 (362,289 states); (iii) *discretization scales gracefully*—the disc. $W = 3$ diamond completes with 98 states versus OOM for the full precise model. Fig. 2 summarizes the estimated maximum verifiable network sizes.

Fig. 2. Estimated maximum verifiable network sizes (2 GB CUDD limit).

Configuration	Chain	Diamond	Notes
Det. precise	>10N	>10N	Minimal state space
Fast precise	5–6N	4N	Per-neuron narrowing
Full precise	4N	<4N	ARP/RRP dominate
Fast disc. $W=3$	>10N	7–8N	$17\times$ reduction/neuron
Full disc. $W=3$	8N	5–6N	Best precision/tractability

7 Conclusion

This paper presented a weight-discretized quotient model abstraction for the formal verification of spiking neural networks. The discretization function δ_W maps continuous synaptic weights to a compact integer range while preserving threshold feasibility (Theorem 1) and preventing spurious spikes (Theorem 2). The core biological properties of LIF neurons—tonic spiking, integrator behaviour, and excitability [5] — are maintained by the threshold-dependent leak factor λ_d .

The topology-dependent scaling analysis demonstrates that the state space reduction compounds exponentially across neurons: approximately $17\times$ per neuron for $W = 3$, enabling verification of networks that are otherwise intractable. Empirical validation across seven canonical topologies confirms the theoretical predictions and identifies BDD memory as the binding practical constraint.

Future directions include *compositional verification* exploiting neuronal archetypes [6], *automated W selection* based on fan-in analysis, extension to *recurrent topologies*, and integrating verification into biologically plausible learning frameworks [13] as modulatory safety constraints.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Bibliography

1. Maass, W.: Networks of Spiking Neurons: The Third Generation of Neural Network Models. *Neural Networks*. 10, 1659–1671 (1997).
2. Nguyen, D.-A., Tran, X.-T., Iacopi, F.: A Review of Algorithms and Hardware Implementations for Spiking Neural Networks. *Journal of Low Power Electronics and Applications*. 11, 23 (2021).
3. Baier, C., Katoen, J.-P.: *Principles of Model Checking*. MIT Press (2008).
4. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: Verification of Probabilistic Real-time Systems. *Proceedings of the 23rd International Conference on Computer Aided Verification*. 585–591 (2011).
5. Maria, E.D., Giusto, C.D., Laversa, L.: Spiking Neural Networks Modelled as Timed Automata with Parameter Learning. *Natural Computing*. 19, 135–155 (2020).
6. Maria, E.D., Bahrami, A., L'Yvonnet, T., Felty, A., Gaffé, D., Ressouche, A., Grammont, F.: On the Use of Formal Methods to Model and Verify Neuronal Archetypes. *Frontiers of Computer Science*. 16, 163404 (2022).
7. Yao, Z., Maria, E.D., Simone, R.D.: Probabilistic Modeling of Spiking Neural Networks with Contract-Based Verification. In: *arXiv preprint arXiv:2506.13340v1* (2025).
8. Hodgkin, A.L., Huxley, A.F.: A Quantitative Description of Membrane Current and Its Application to Conduction and Excitation in Nerve. *The Journal of Physiology*. 117, 500–544 (1952).
9. Katoen, J.-P., Kemna, T., Zapreev, I.S., Jansen, D.N.: Bisimulation Minimisation for Probabilistic Systems. *Logical Methods in Computer Science*. 12, (2016).
10. Bekolay, T., Bergstra, J., Hunsberger, E., DeWolf, T., Stewart, T.C., Rasmussen, D., Choo, X., Voelker, A., Eliasmith, C.: Nengo: A Python Tool for Building Large-Scale Functional Brain Models. *Frontiers in Neuroinformatics*. 7, 48 (2014).
11. Stimberg, M., Brette, R., Goodman, D.F.M.: Brian 2, an Intuitive and Efficient Neural Simulator. *eLife*. 8, e47314 (2019).
12. Gewaltig, M.-O., Diesmann, M.: NEST (NEural Simulation Tool). *Scholarpedia*. 2, 1430 (2007).
13. Bellec, G., Scherr, F., Hajek, A., Salaj, D., Legenstein, R., Maass, W.: Eligibility Traces Provide a Data-Inspired Alternative to Backpropagation Through Time. In: *Real Neurons Hidden Units: Future Directions at the Intersection of Neuroscience and Artificial Intelligence @ NeurIPS* (2019).